TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC CUỐI KÌ

MẬT MÃ ỨNG DỤNG

NGHIÊN CỨU KỸ THUẬT XÁC THỰC HASH FUNCTION VÀ XÂY DỰNG ỨNG DỤNG MINH HỌA

Giáo viên hướng dẫn:

Sinh viên thực hiện:

Huỳnh Thiên Hạo- 23162021

Cao Đăng Huy - 23162028

Võ Gia Huân – 23162027

Trương Nguyễn Minh Hậu -23162022

Mã học phần: CRYP331980_02

TP. HỒ CHÍ MINH - NĂM 2025

BẢNG PHÂN CÔNG NHIỆM VỤ

THỨ TỰ	HỌ TÊN	NHIỆM VỤ	KẾT QUẢ	KÝ TÊN
1	Huỳnh Thiên Hạo	- Quản lý nhóm, nghiên cứu lý thuyết Hash Function. - Viết báo cáo. - Hỗ trợ kiểm thử. - Triển khai chữ ký số (RSA + SHA256).	100/100	
2	Cao Đăng Huy	- Tổng hợp tài liệu. - Viết báo cáo. - Hỗ trợ kiểm thử.	100/100	
3	Võ Gia Huân	- Thiết kế database Firebase - Xử lý chức năng đăng ký/đăng nhập.	100/100	
4	Trương Nguyễn Minh Hậu	- Lập trình giao diện web đăng ký/đăng nhập. - Kiểm thử	100/100	

KẾ HOẠCH THỰC HIỆN

STT	Thời gian	Công việc	Ghi chú
1	06/03 - 20/03/2025	- Nghiên cứu lý thuyết Hash Function - Lập đề cương chi tiết	Đã hoàn thành
2	21/03 - 03/04/2025	- Thiết kế database Firebase - Xây dựng chức năng đăng ký/đăng nhập	Đã hoàn thành
3	04/04 - 17/04/2025	- Triển khai chữ ký số (RSA + SHA256) - Kiểm thử ứng dụng	Đã hoàn thành
4	18/04 - 30/04/2025	- Hoàn thiện báo cáo - Tối ưu giao diện người dùng	Đã hoàn thành
5	01/05 - 05/05/2025	- Mở rộng hỗ trợ định dạng file PDF/DOCX - Chuẩn bị báo cáo cuối kỳ	Đã hoàn thành

MỤC LỤC

PHẦN MỞ ĐẦU 5
1. Tính cấp thiết của đề tài 5
2. Mục đích của đề tài 5
3. Cách tiếp cận và phương pháp nghiên cứu 6
PHẦN NỘI DUNG 8
CHƯƠNG 1: CƠ SỞ LÍ THUYẾT HÀM BĂM (HASHFUNCTION) 8
1.1. Khái niệm 8
1.2. Tính chất cơ bản của hàm băm 9
1.3. Một số hàm băm phổ biến 10
1.4. Ứng dụng của hàm băm 16
CHƯƠNG 2: THIẾT KẾ VÀ CÀI ĐẶT ỨNG DỤNG MINH HỌA 21
2.1. Yêu cầu hệ thống 21
2.2. Mô hình kiến trúc hệ thống 21
2.3. Giao diện người dùng 22
2.4. Mô tả chức năng chính 23
2.4.1. Đăng ký/ đăng nhập tài khoản người dùng (lưu trữ mật khẩu) 23
2.4.2. Chữ kí số 24
CHƯƠNG 3: TỔNG KẾT 26
3.1. Kết quả đạt được 26
3.1.1. Về mặt lí thuyết 26
3.1.2. Về mặt ứng dụng 26
3.1.3. Hạn chế 27
3.2. Định hướng phát triển trong tương lai 27
3.2.1. Kế hoạch ngắn hạn 27
3.2.2. Kế hoạch dài hạn 28
TÀI LIỆU THAM KHẢO 29

PHẦN MỞ ĐẦU

1. Tính cấp thiết của đề tài

Trong kỷ nguyên số hóa, nơi dữ liệu được ví như "vàng", bảo mật thông tin không còn là một tùy chọn mà là yếu tố sống còn cho mỗi cá nhân, tổ chức và doanh nghiệp. Giữa "ma trận" dữ liệu khổng lồ ấy, Hash Function (hàm băm) nổi lên như một "vệ sĩ" thầm lặng nhưng vô cùng mạnh mẽ, đóng vai trò then chốt trong việc mã hóa và xác thực danh tính người dùng.

Hãy hình dung, thay vì phơi bày mật khẩu của bạn dưới dạng văn bản thuần túy (plaintext) đầy rủi ro, các hệ thống ngày nay đã khóa kín chúng bằng hàm băm, bổ sung thêm một lớp "muối" (salt) phức tạp. Đây chính là lá chắn đầu tiên, giảm thiểu tối đa nguy cơ bị xâm nhập nếu chẳng may dữ liệu bị đánh cắp.

Nhưng "sức mạnh" của hàm băm không chỉ dừng lại ở đó. Bạn đã bao giờ tự hỏi làm thế nào một tệp tin dung lượng lớn vẫn nguyên sau khi được tải xuống từ một nơi khác? Bí mật nằm ở checksum, một "dấu vân tay kỹ thuật số" được tạo ra bởi hàm băm. Chỉ cần một thay đổi nhỏ nhất trong dữ liệu, "dấu vân tay" này sẽ lập tức khác biệt, giúp bạn dễ dàng phát hiện ra sự can thiệp bất thường.

Tuy nhiên, với sự phát triển của các phương pháp tấn công như brute-force, collision attack, rainbow table, nhiều thuật toán băm cũ như (MD5, SHA-1) đã bị crack, nên đòi hỏi phải nghiên cứu sâu hơn về các kỹ thuật băm hiện đại như (SHA-3, BLAKE3, Bcrypt,…) và những phương pháp xác thực an toàn hơn.

Do đó, việc nghiên cứu kỹ thuật xác thực Hash Function không chỉ có ý nghĩa học thuật mà còn mang tính ứng dụng cao, góp phần nâng cao nhận thức và giải pháp bảo mật trong thực tế.

2. Mục đích của đề tài

Đề tài nhóm tập trung vào các mục tiêu nghiên cứu cụ thể sau:

Nghiên cứu lý thuyết: Tìm hiểu sâu về nguyên lý hoạt động và các đặc tính quan trọng của hàm băm (tính một chiều, khả năng chống va chạm). Đồng thời, so sánh các thuật toán băm phổ biến hiện nay, bao gồm MD5, SHA-1, SHA-256 và SHA-3.

Phân tích rủi ro bảo mật: Đánh giá các phương pháp tấn công tiềm ẩn nhằm vào hàm băm, chẳng hạn như tấn công sinh nhật (birthday attack) và tấn công mở rộng độ dài (length extension attack). Nghiên cứu các biện pháp phòng chống hiệu quả như sử dụng salt và kỹ thuật kéo dài khóa (key stretching).

Xây dựng ứng dụng minh họa: Triển khai một hệ thống demo thực tế để làm rõ tính ứng dụng của hàm băm. Các ví dụ bao gồm công cụ kiểm tra hash của tập tin, hệ thống xác thực mật khẩu an toàn và ứng dụng chữ ký số.

Đề xuất hướng phát triển: Đưa ra các gợi ý về các thuật toán băm mới nổi (ví dụ: BLAKE3, Argon2) và các xu hướng bảo mật liên quan trong tương lai.

Kết quả nghiên cứu dự kiến sẽ cung cấp một tài liệu tham khảo giá trị cho sinh viên, các kỹ sư an ninh mạng và nhà phát triển phần mềm quan tâm đến lĩnh vực bảo mật thông tin.

3. Cách tiếp cận và phương pháp nghiên cứu

Để đạt được các mục tiêu đã đề ra, nhóm nghiên cứu sẽ tiếp cận một cách hệ thống thông qua ba phương pháp chính.

Đầu tiên, phương pháp nghiên cứu lý thuyết tập trung vào việc tổng hợp và phân tích sâu các tài liệu chuyên ngành uy tín về mật mã học, tiêu chuẩn kỹ thuật từ các tổ chức như NIST và IETF, cùng các bài báo khoa học liên quan đến hàm băm. Qua đó, nhóm sẽ làm rõ nguyên lý hoạt động và so sánh các đặc điểm then chốt của các thuật toán băm phổ biến.

Thứ hai, phương pháp phân tích và đánh giá sẽ được triển khai bằng cách so sánh hiệu suất giữa các thuật toán khác nhau và đặc biệt là tiến hành kiểm tra tính bảo mật thông qua các kịch bản tấn công giả lập, sử dụng các công cụ chuyên dụng để minh họa các lỗ hổng tiềm ẩn.

Cuối cùng, phương pháp thực nghiệm sẽ cụ thể hóa lý thuyết và đánh giá thông qua việc xây dựng các ứng dụng minh họa thực tế, chẳng hạn như công cụ tính toán và kiểm tra hash file bằng Python, cũng như thiết kế hệ thống đăng nhập web an toàn sử

dụng các kỹ thuật bảo mật mật khẩu tiên tiến. Sự kết hợp nhuần nhuyễn giữa ba phương pháp này sẽ đảm bảo nhóm có thể đạt được các mục tiêu nghiên cứu một cách toàn diện.

PHẦN NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÍ THUYẾT HÀM BĂM (HASHFUNCTION)

Trong thế giới số hóa ngày nay, việc bảo vệ thông tin trở nên quan trọng hơn bao giờ hết. Từ việc đăng nhập vào tài khoản cá nhân, xác minh tính toàn vẹn của một phần mềm tải về, đến việc thực hiện các giao dịch trực tuyến an toàn, chúng ta đều phụ thuộc vào các công nghệ bảo mật mạnh mẽ. Một trong những công nghệ nền tảng và không thể thiếu đó chính là hàm băm (hash function).

Hãy thử tưởng tượng bạn muốn gửi một tài liệu quan trọng cho đối tác và muốn đảm bảo rằng nó không bị thay đổi trên đường truyền. Hoặc khi bạn tạo một tài khoản trực tuyến, làm thế nào hệ thống có thể lưu trữ mật khẩu của bạn một cách an toàn mà không ai có thể đọc được? Các vấn đề đó điều được giải quyết bằng hàm băm.

1.1. Khái niệm

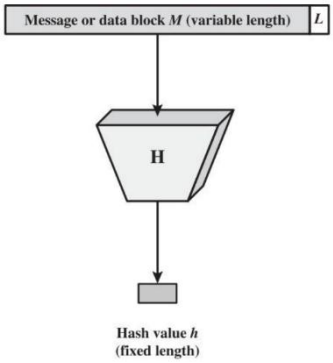
Về cơ bản, hàm băm là một hàm toán học đặc biệt, nhận đầu vào là một thông điệp hoặc dữ liệu có kích thước bất kỳ và tạo ra một đầu ra có kích thước cố định(tùy thuộc vào thuật toán được sử dụng), thường được gọi là giá trị băm (hash value), mã băm (hash code), hoặc digest đại diện cho thông điệp ban đầu được nhập vào.

Khác với các thuật toán mã hóa, có thể mã hóa thông tin và giải mã ngược lại thông tin ban đầu thì hàm băm mang tính một chiều.

Tnh chất "một chiều" của hàm băm là yếu tố then chốt, ngụ ý rằng việc đảo ngược quá trình để tìm lại dữ liệu gốc từ giá trị băm là vô cùng khó khăn, trên thực tế là bất khả thi với các phương pháp tính toán thông thường.

Để dễ hình dung, bạn có thể xem hàm băm như một chiếc máy xay sinh tố đặc biệt. Bất kể bạn bỏ vào đó loại trái cây, rau củ gì (dữ liệu đầu vào), máy sẽ luôn tạo ra một ly sinh tố có dung tích nhất định (giá trị băm có kích thước cố định) với hương vị đặc trưng (giá trị băm là duy nhất cho đầu vào đó). Và việc bạn lấy lại nguyên vẹn hình dạng các loại trái cây rau củ đó như ban đầu là điều không thể.

Chính đặc điểm này đã mở ra vô vàn ứng dụng quan trọng của hàm băm trong lĩnh vực an toàn thông tin, từ việc bảo vệ mật khẩu người dùng, đảm bảo tính toàn vẹn của dữ liệu truyền tải, đến việc tạo nền tảng cho các công nghệ tiên tiến như chữ ký số và blockchain. Giá trị băm, dù ngắn gọn, lại mang trong mình một "dấu vân tay" kỹ thuật số độc đáo cho dữ liệu đầu vào, cho phép chúng ta xác minh tính xác thực và phát hiện mọi sự thay đổi dù là nhỏ nhất.



Input – M có kích thước bất kỳ.

Output – giá trị h có kích thước cố định, ngắn.

H(x) – hàm một chiều (“Khó để tính nghịch đảo”)

1.2. Tính chất cơ bản của hàm băm

Tính chống tiền ảnh (Preimage resistant – one-way property)

Cho một giá trị băm h, việc tìm bất kỳ đầu vào M nào sao cho H(M) = h là bất khả thi về mặt tính toán. Điều này có nghĩa là rất khó để đảo ngược quá trình băm để tìm lại dữ liệu gốc từ giá trị băm. Tính năng này bảo vệ chống lại kẻ tấn công cố gắng xác định vị trí đầu vào chỉ với giá trị băm.

Tính chống tiền ảnh thứ hai (Second preimage resistant – weak collision resistant – Tính chống trùng yếu)

Cho một đầu vào M1 cụ thể, việc tìm một đầu vào khác M2 (M2 ≠ M1) sao cho H(M1) = H(M2) là bất khả thi về mặt tính toán. Tính năng này của hàm băm bảo vệ chống lại kẻ tấn công muốn thay thế một giá trị mới cho giá trị đầu vào ban đầu và băm, nhưng chỉ giữ giá trị đầu vào và băm của nó.

Tính chống trùng mạnh (Strong Collision resistant)

Tính năng này nói rằng rất khó để xác định hai đầu vào khác nhau của bất kỳ độ dài nào tạo ra cùng một hàm băm. Đặc điểm này còn được gọi là hàm băm không va chạm. Nói cách khác, đối với hàm băm H, rất khó để xác định hai đầu vào riêng biệt x

và y sao cho H(x) = H(y). Chức năng băm không thể không va chạm vì nó là hàm nén với độ dài băm tập hợp. Điều kiện không va chạm chỉ đơn giản chỉ ra rằng những va chạm này sẽ khó xác định vị trí. Đặc điểm này làm cho một kẻ tấn công rất khó xác định hai giá trị đầu vào có cùng băm. Hơn nữa, một hàm băm là khả năng chống hình ảnh thứ hai nếu nó có khả năng chống va chạm.

(Nghịch lý ngày sinh – Birthday paradox)

Bài toán đặt ra là: "Trong một phòng có M sinh viên, xác suất để có 2 người cùng ngày sinh là bao nhiêu? (giả sử 1 năm có 365 ngày khác nhau)"

Theo nguyên lý chuồng bồ câu (Dirichlet):

Cần ít nhất 366 người để đảm bảo chắc chắn có 2 người cùng ngày sinh (vì có 365 ngày → nếu có 366 người thì chắc chắn trùng).

→ Với 30 người, trực giác khiến ta nghĩ xác suất trùng là rất nhỏ.

Theo xác suất thống kê toán học:

Khi số người M thỏa mãn bất đẳng thức:

$$M\,(M-1) \geq 2 \times 365 \times \ln(2)$$

thì xác suất có ít nhất 2 người cùng ngày sinh lớn hơn 50%.

Chỉ cần 23 người đã đủ để vượt mốc 50%. Đây là nghịch lý ngày sinh – vì nó mâu thuẫn với trực giác ban đầu.

Trong mật mã học, điều này cho thấy khả năng trùng Hash (collision) cao hơn ta tưởng nếu không thiết kế cẩn thận.

Vì vậy cần dùng hàm băm có tính chống trùng mạnh, tức hàm băm có độ dài đủ lớn.

1.3. Một số hàm băm phổ biến

Message Digest (MD)

Trong nhiều năm, MD5 từng là một trong những hàm băm được sử dụng phổ biến nhất trong lĩnh vực công nghệ thông tin.

Các hàm băm như MD2, MD4, MD5 và MD6 đều thuộc họ Message Digest (MD). Trong đó, MD5 được tiêu chuẩn hóa trong RFC 1321 và tạo ra giá trị băm dài 128 bit.

MD5 từng được sử dụng rộng rãi để kiểm tra tính toàn vẹn của dữ liệu, đặc biệt trong việc tải xuống phần mềm từ Internet.

Các trang web thường cung cấp chuỗi băm MD5 đi kèm với tệp tin, giúp người dùng kiểm tra xem tệp đã tải về có bị lỗi hoặc bị chỉnh sửa trong quá trình truyền tải hay không bằng cách so sánh với giá trị băm gốc.

Vào năm 2004, các chuyên gia bảo mật đã chứng minh được rằng có thể tạo ra va chạm trong MD5, tức là hai dữ liệu khác nhau có thể sinh ra cùng một giá trị băm.

Điều đáng chú ý là việc tìm ra va chạm có thể thực hiện được chỉ trong vòng chưa đầy một giờ nếu sử dụng cụm máy tính mạnh.

Kể từ đó, MD5 bị coi là không còn an toàn, đặc biệt trong các hệ thống yêu cầu tính bảo mật cao như xác thực chữ ký số, mã hóa, hay chứng thực dữ liệu.

Secure Hash Function (SHA)

Họ thuật toán băm SHA (Secure Hash Algorithm) cấu thành một tập hợp các hàm băm mật mã hóa được thiết kế bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST). Mặc dù thuộc cùng một dòng họ, các thuật toán SHA-0, SHA-1, SHA-2 và SHA-3 thể hiện sự khác biệt đáng kể trong kiến trúc nội tại của chúng, phản ánh sự tiến hóa trong hiểu biết về các yêu cầu an ninh và các lỗ hổng tiềm ẩn.

Năm 1993, NIST giới thiệu SHA-0, thuật toán băm đầu tiên thuộc họ này, với độ dài đầu ra cố định là 160 bit. Tuy nhiên, do những nhược điểm cấu trúc được phát hiện sau đó, SHA-0 không đạt được sự chấp nhận rộng rãi trong cộng đồng mật mã học.

Để khắc phục những hạn chế được nhận thấy ở SHA-0, SHA-1 đã được phát triển và công bố vào năm 1995. Vẫn duy trì độ dài băm 160 bit, SHA-1 đã tích hợp các cải tiến thiết kế nhằm tăng cường khả năng chống lại các phân tích mật mã. Thuật toán này nhanh chóng trở thành một tiêu chuẩn de facto và được triển khai rộng rãi trong nhiều

giao thức và ứng dụng bảo mật, bao gồm cả giao thức Secure Socket Layer (SSL), đóng vai trò thiết yếu trong việc bảo mật truyền thông trên mạng.

Tuy nhiên, vào năm 2005, những nghiên cứu đột phá trong lĩnh vực mật mã phân tích đã công bố các kỹ thuật thực tế cho việc tìm kiếm xung đột (collision) trong SHA-1. Sự tồn tại của các cuộc tấn công xung đột hiệu quả làm dấy lên những quan ngại nghiêm trọng về tính toàn vẹn và độ tin cậy lâu dài của SHA-1 trong các ứng dụng nhạy cảm với bảo mật.

Nhằm cung cấp các giải pháp băm mạnh mẽ hơn, họ SHA-2 đã được phát triển, bao gồm các biến thể SHA-224, SHA-256, SHA-384 và SHA-512. Các thuật toán này khác nhau chủ yếu về độ dài của giá trị băm đầu ra, lần lượt là 224, 256, 384 và 512 bit. Mặc dù SHA-2 chia sẻ một số nguyên tắc thiết kế cơ bản với SHA-1, kiến trúc của nó đã được tăng cường đáng kể để chống lại các kỹ thuật tấn công đã biết. Tính đến thời điểm hiện tại, chưa có bất kỳ cuộc tấn công hiệu quả nào được công bố nhằm vào các hàm băm SHA-2.

Mặc dù SHA-2 được coi là một họ hàm băm an toàn, sự tương đồng về mặt thiết kế với SHA-1 đã thúc đẩy NIST khởi xướng một cuộc thi nhằm tìm kiếm các thiết kế hàm băm cạnh tranh và đa dạng hơn. Mục tiêu là để phát triển một tiêu chuẩn băm thế hệ mới, độc lập về mặt kiến trúc với các thuật toán SHA trước đó, nhằm giảm thiểu rủi ro từ các lỗ hổng thiết kế tiềm ẩn có thể ảnh hưởng đến toàn bộ họ thuật toán.

Kết quả của quá trình tuyển chọn này là việc NIST công bố thuật toán Keccak là tiêu chuẩn SHA-3 vào tháng 10 năm 2012. Keccak dựa trên một nguyên lý thiết kế hoàn toàn khác biệt, được gọi là cấu trúc sponge, mang lại hiệu suất cao và khả năng chống chịu mạnh mẽ đối với nhiều lớp tấn công mật mã. Sự ra đời của SHA-3 đánh dấu một bước tiến quan trọng trong việc đa dạng hóa các công cụ mật mã nền tảng, tăng cường khả năng phục hồi của các hệ thống bảo mật trước những tiến bộ không ngừng trong lĩnh vực tấn công mật mã học.

CityHash

Một thuật toán băm không dùng cho bảo mật, được Google phát triển để thực hiện các phép băm nhanh và hiệu quả cho dữ liệu lớn. Nó được tối ưu hóa cho các bộ vi

xử lý hiện đại và hoạt động tốt trên cả kiến trúc 32-bit và 64-bit. CityHash chủ yếu được sử dụng trong các ứng dụng yêu cầu tốc độ cao, chẳng hạn như tìm kiếm, phân tán dữ liệu hoặc bảng băm.

BLAKE2

BLAKE2 là một hàm băm mật mã hóa được thiết kế để đạt được tốc độ cao và tính bảo mật vững chắc, đồng thời được xem là một sự cải tiến đáng kể so với SHA-3. Nhờ hiệu suất vượt trội, BLAKE2 đã tìm thấy nhiều ứng dụng quan trọng, đặc biệt trong các lĩnh vực đòi hỏi tốc độ băm nhanh chóng như khai thác tiền điện tử (cryptocurrency mining). Họ hàm BLAKE2 bao gồm hai biến thể chính:

BLAKE2b: Được tối ưu hóa cho các hệ thống máy tính 64-bit, BLAKE2b có khả năng tạo ra các giá trị băm với độ dài linh hoạt lên đến 512 bits. Điều này cho phép người dùng tùy chỉnh mức độ bảo mật và hiệu suất tùy theo yêu cầu cụ thể của ứng dụng.

BLAKE2s: Được thiết kế để hoạt động hiệu quả trên các hệ thống nhỏ hơn, chẳng hạn như máy tính 8-bit đến 32-bit, BLAKE2s tạo ra các giá trị băm với độ dài tối đa 256 bits. Sự tối ưu hóa này giúp BLAKE2s trở thành lựa chọn phù hợp cho các thiết bị di động, hệ thống nhúng và các môi trường tài nguyên hạn chế.

CRC (Cyclic Redundancy Check)

Cyclic Redundancy Check (CRC) là một kỹ thuật phát hiện lỗi mạnh mẽ và hiệu quả, được sử dụng rộng rãi trong các hệ thống truyền thông và lưu trữ dữ liệu để đảm bảo tính toàn vẹn của thông tin. Cơ chế hoạt động của CRC dựa trên việc thêm một giá trị kiểm tra, thường được gọi là mã dư thừa vòng (checksum) hoặc đa thức kiểm tra khung (frame check sequence - FCS), vào cuối mỗi khối dữ liệu (message) trước khi truyền đi. Giá trị checksum này được tính toán dựa trên nội dung của thông điệp gốc thông qua các phép toán đại số trên trường hữu hạn (finite field), thường là trường Galois GF(2).

Cơ chế Hoạt động Chi tiết:

Quá trình tạo mã CRC bao gồm việc coi chuỗi bit của thông điệp như một đa thức hệ số nhị phân. Đa thức này sau đó được chia cho một đa thức sinh (generator polynomial) đã được thống nhất trước giữa bên gửi và bên nhận. Đa thức sinh là một chuỗi bit (hoặc đa thức nhị phân) có bậc k, và nó quyết định khả năng phát hiện các loại lỗi khác nhau của mã CRC. Số dư của phép chia đa thức này chính là giá trị checksum (FCS) được thêm vào cuối thông điệp.

Khi dữ liệu được nhận, bên nhận thực hiện lại phép chia đa thức tương tự trên toàn bộ chuỗi bit đã nhận (bao gồm cả thông điệp gốc và checksum). Nếu không có lỗi xảy ra trong quá trình truyền, số dư của phép chia tại bên nhận sẽ là một chuỗi bit toàn số 0. Ngược lại, nếu số dư khác 0, điều này chỉ ra rằng đã có lỗi xảy ra trong quá trình truyền dữ liệu.

Các Loại CRC Phổ Biến:

Có nhiều biến thể của CRC được sử dụng trong các tiêu chuẩn và ứng dụng khác nhau, chủ yếu khác nhau về độ dài của đa thức sinh và do đó, độ dài của giá trị checksum. Một số CRC phổ biến bao gồm:

CRC-16: Sử dụng đa thức sinh bậc 16, tạo ra checksum 16-bit. Thường được sử dụng trong các giao thức như HDLC và PPP.

CRC-32: Sử dụng đa thức sinh bậc 32, tạo ra checksum 32-bit. Rất phổ biến trong Ethernet (IEEE 802.3), ZIP và PNG.

CRC-8: Sử dụng đa thức sinh bậc 8, tạo ra checksum 8-bit. Thường được sử dụng trong các ứng dụng có yêu cầu về overhead thấp hơn.

CRC-64: Sử dụng đa thức sinh bậc 64, tạo ra checksum 64-bit. Cung cấp khả năng phát hiện lỗi mạnh mẽ hơn và được sử dụng trong một số hệ thống lưu trữ và truyền thông tốc độ cao.

Hàm băm Bcrypt

Bcrypt là một hàm băm mật khẩu (password hashing function) phổ biến và được đánh giá cao về độ an toàn. Nó được thiết kế đặc biệt để băm mật khẩu một chiều, nghĩa

là rất khó (về mặt tính toán là bất khả thi trong thực tế) để đảo ngược quá trình băm và khôi phục lại mật khẩu gốc từ giá trị băm.

Bcrypt được xây dựng dựa trên thuật toán mã hóa khối Blowfish, nhưng đã được điều chỉnh để phù hợp với mục đích băm mật khẩu. Nó sử dụng một biến thể của Blowfish với một số thay đổi quan trọng.

Một trong những yếu tố quan trọng làm nên độ an toàn của bcrypt là việc sử dụng "salt". Salt là một chuỗi dữ liệu ngẫu nhiên được thêm vào mỗi mật khẩu trước khi băm. Từ đó, ngăn chặn tấn công dựa trên bảng cầu vồng (rainbow table attacks), tăng cường độ phức tạp của việc tấn công brute-force.

Bcrypt cho phép điều chỉnh một tham số gọi là "cost factor" (hoặc "work factor"). Tham số này quyết định độ phức tạp tính toán của quá trình băm. Giá trị cost factor càng cao, thời gian cần thiết để băm một mật khẩu càng lâu, và do đó, thời gian cần thiết để thực hiện một cuộc tấn công brute-force thành công cũng tăng lên theo hàm mũ.

Tính thích ứng theo thời gian: Một ưu điểm quan trọng của cost factor là nó có thể được tăng lên theo thời gian khi sức mạnh tính toán của máy tính tăng lên. Điều này giúp duy trì mức độ an toàn của các mật khẩu đã băm trong tương lai. Khi một ứng dụng muốn tăng cường bảo mật, họ có thể băm lại mật khẩu của người dùng với cost factor cao hơn.

Quy trình băm mật khẩu với bcrypt:

Tạo salt ngẫu nhiên: Một chuỗi byte ngẫu nhiên được tạo ra để làm salt cho mật khẩu.

Kết hợp mật khẩu và salt: Mật khẩu và salt được kết hợp lại với nhau.

Thực hiện hàm băm bcrypt: Hàm băm bcrypt được áp dụng lên chuỗi đã kết hợp, sử dụng cost factor đã chọn. Quá trình này bao gồm nhiều vòng lặp của một biến thể của thuật toán Blowfish.

Lưu trữ giá trị băm và salt: Giá trị băm kết quả, cùng với salt đã sử dụng (thường được nhúng trực tiếp vào chuỗi băm), được lưu trữ trong cơ sở dữ liệu.

Quy trình xác thực mật khẩu với bcrypt:

Lấy giá trị băm và salt đã lưu trữ: Khi người dùng cố gắng đăng nhập, hệ thống sẽ truy xuất giá trị băm và salt tương ứng với tên người dùng của họ từ cơ sở dữ liệu.

Kết hợp mật khẩu người dùng nhập và salt: Mật khẩu mà người dùng vừa nhập vào sẽ được kết hợp với salt đã được lưu trữ.

Băm lại mật khẩu đã nhập: Hàm băm bcrypt (với cùng cost factor đã sử dụng khi tạo băm ban đầu) sẽ được áp dụng lên chuỗi đã kết hợp này.

So sánh các giá trị băm: Giá trị băm mới được tạo ra sẽ được so sánh với giá trị băm đã lưu trữ. Nếu chúng khớp nhau, mật khẩu được coi là chính xác.

Tóm lại, bcrypt là một lựa chọn tuyệt vời để băm mật khẩu nhờ vào việc sử dụng salt, tham số cost factor có thể điều chỉnh và thiết kế dựa trên một thuật toán mã hóa mạnh mẽ. Nó được khuyến nghị sử dụng trong hầu hết các ứng dụng cần lưu trữ mật khẩu người dùng một cách an toàn.

1.4. Ứng dụng của hàm băm

Bảo Vệ Lưu Trữ Mật Khẩu

Các hàm băm mật mã hóa đóng vai trò then chốt trong việc bảo vệ thông tin nhạy cảm như mật khẩu người dùng. Thay vì lưu trữ mật khẩu ở dạng văn bản gốc (plaintext), hầu hết các quy trình đăng nhập hiện đại đều sử dụng hàm băm để tạo ra một giá trị đại diện duy nhất (hash value) cho mỗi mật khẩu và lưu trữ các giá trị băm này trong cơ sở dữ liệu.

Cấu trúc điển hình của tệp mật khẩu là một bảng chứa các cặp giá trị có định dạng (ID người dùng, h(P)), trong đó h(P) là giá trị băm của mật khẩu P tương ứng với người dùng có ID đó. Ngay cả khi một kẻ tấn công xâm nhập thành công vào hệ thống và có quyền truy cập vào tệp mật khẩu, tất cả những gì chúng có thể thấy chỉ là các giá trị băm vô nghĩa.

Tính chất kháng tiền ảnh (pre-image resistance) của các hàm băm mật mã hóa đảm bảo rằng với một giá trị băm cho trước, việc tìm ra thông điệp gốc (trong trường hợp này là mật khẩu) là cực kỳ khó khăn về mặt tính toán, được coi là bất khả thi trong thực tế với các hàm băm mạnh mẽ. Do đó, kẻ tấn công không thể trực tiếp sử dụng các giá trị băm này để đăng nhập vào tài khoản hoặc khôi phục mật khẩu gốc.

Tuy nhiên, cần lưu ý rằng chỉ sử dụng hàm băm thôi là chưa đủ để bảo vệ hoàn toàn mật khẩu. Các kỹ thuật tấn công như tấn công từ điển (dictionary attack) và tấn công bảng cầu vồng (rainbow table attack) có thể được sử dụng để cố gắng tìm ra mật khẩu gốc từ các giá trị băm phổ biến. Để đối phó với các mối đe dọa này, các phương pháp bảo vệ bổ sung như sử dụng salt (một chuỗi ngẫu nhiên được thêm vào mật khẩu trước khi băm) và các hàm băm có khả năng làm chậm quá trình tính toán (ví dụ: bcrypt, Argon2) là rất cần thiết.

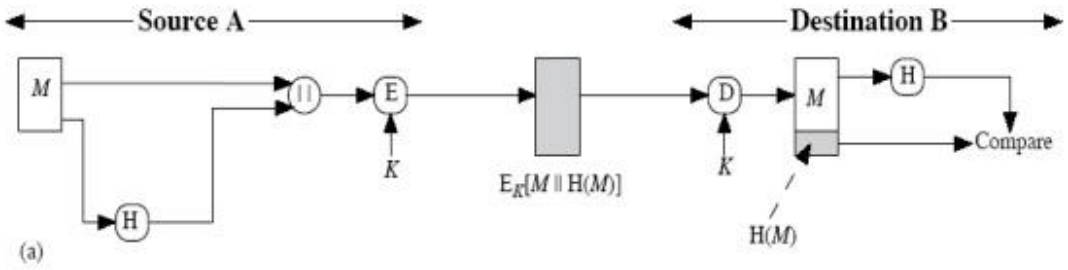
Kiểm Tra Tính Toàn Vẹn Dữ Liệu

Các hàm băm cũng được ứng dụng rộng rãi trong việc kiểm tra tính toàn vẹn dữ liệu. Bằng cách tạo ra một giá trị kiểm tra (checksum) duy nhất từ nội dung của một tệp dữ liệu, người dùng có thể xác minh rằng tệp đó không bị thay đổi hoặc hư hỏng trong quá trình lưu trữ hoặc truyền tải.

Khi một tệp được tạo hoặc truyền đi, một giá trị băm (checksum) sẽ được tính toán và thường được lưu trữ riêng hoặc đi kèm với tệp. Khi người dùng muốn kiểm tra tính toàn vẹn của tệp, họ sẽ tính toán lại giá trị băm từ tệp hiện tại và so sánh nó với giá trị băm ban đầu. Nếu hai giá trị băm trùng khớp, điều này cho thấy dữ liệu có khả năng cao vẫn còn nguyên vẹn so với thời điểm ban đầu.

Tuy nhiên, cần nhấn mạnh rằng việc kiểm tra tính toàn vẹn bằng hàm băm không đảm bảo tính xác thực (authenticity) của tệp. Một kẻ tấn công có khả năng sửa đổi toàn bộ nội dung của tệp và sau đó tính toán một giá trị băm mới phù hợp với nội dung đã bị thay đổi. Nếu giá trị băm mới này được gửi đến người nhận mà không có bất kỳ dấu hiệu nào cho thấy sự can thiệp, người nhận sẽ không thể phát hiện ra sự thay đổi nếu chỉ dựa vào việc so sánh giá trị băm. Do đó, việc kiểm tra tính toàn vẹn bằng hàm băm chỉ thực sự hiệu quả khi người dùng tin tưởng vào nguồn gốc ban đầu của tệp và giá trị băm

đi kèm. Để đảm bảo tính xác thực, cần sử dụng các kỹ thuật khác như chữ ký số (digital signatures).



Cơ chế chứng thực đơn giản

Chữ kí số (Digital Signatures)

Trong hệ thống chữ ký số, giá trị băm (hash) của một thông điệp thường được mã hóa bằng khóa bí mật (private key) của người gửi. Quá trình này tạo ra một chữ ký số (digital signature), gắn liền với thông điệp gốc.

Sau khi nhận được thông điệp và chữ ký số đi kèm, bất kỳ ai biết khóa công khai (public key) của người gửi đều có thể:

Tính toán lại giá trị băm từ thông điệp gốc.

Giải mã chữ ký số bằng khóa công khai để lấy ra giá trị băm ban đầu do người gửi tạo ra.

So sánh hai giá trị băm để kiểm tra xem thông điệp có bị thay đổi trong quá trình truyền hay không.

Nếu hai giá trị băm khớp nhau, điều đó chứng tỏ rằng:

Thông điệp không bị chỉnh sửa.

Chữ ký thực sự được tạo ra bởi chủ sở hữu của khóa bí mật, vì chỉ họ mới có thể tạo ra chữ ký phù hợp với khóa công khai đó.

Dùng nhận diện xâm hại (intrusion detection) và nhận diện virus (virus detection).

Nhận diện xâm hại thường được thực hiện bởi các hệ thống IDS (Intrusion Detection Systems), có nhiệm vụ giám sát lưu lượng mạng hoặc hành vi hệ thống để phát hiện những hành động đáng ngờ, chẳng hạn như truy cập trái phép, tấn công từ chối

dịch vụ (DoS) hay quét cổng. IDS có thể hoạt động theo phương pháp dựa trên chữ ký (so sánh với mẫu tấn công đã biết) hoặc phát hiện bất thường (so với hành vi chuẩn). Trong khi đó, nhận diện virus là quá trình quét và phân tích các tập tin hoặc ứng dụng trong hệ thống nhằm phát hiện phần mềm độc hại như virus, trojan hay ransomware.

Phần mềm chống virus cũng sử dụng phương pháp dựa trên chữ ký, phân tích hành vi (heuristic), và thậm chí môi trường cách ly (sandbox) để kiểm tra mã độc chưa từng biết đến. Cả hai công nghệ đều đóng vai trò quan trọng trong việc đảm bảo an toàn thông tin: một bên bảo vệ hệ thống ở cấp độ mạng, trong khi bên còn lại bảo vệ ở cấp độ tập tin và phần mềm.

Việc triển khai đồng thời cả IDS và phần mềm diệt virus giúp xây dựng một lớp phòng thủ vững chắc hơn trước các mối đe dọa an ninh mạng hiện đại.

Xây dựng hàm ngẫu nhiên giả (pseudorandom function - PRF) hoặc phát sinh số ngẫu nhiên giả (pseudorandom number generator - PRNG)

Trong lĩnh vực mật mã học, hàm ngẫu nhiên giả (Pseudorandom Function – PRF) và bộ phát sinh số ngẫu nhiên giả (Pseudorandom Number Generator – PRNG) đóng vai trò quan trọng trong việc đảm bảo tính bảo mật và ngẫu nhiên trong các hệ thống mã hóa.

Một PRF là một hàm nhận vào khóa bí mật và đầu vào tùy ý, sau đó tạo ra một đầu ra không thể phân biệt được với một hàm hoàn toàn ngẫu nhiên nếu không biết khóa. PRF thường được dùng trong các giao thức như xác thực, tạo mã kiểm tra tính toàn vẹn (MAC), hoặc trong quá trình mã hóa các gói tin dữ liệu.

Trong khi đó, PRNG được sử dụng để tạo ra một chuỗi các số có vẻ ngẫu nhiên, xuất phát từ một giá trị ban đầu gọi là "seed". PRNG thường được dùng trong sinh khóa mật mã, tạo số ngẫu nhiên trong thuật toán sinh chữ ký số, hoặc trong quy trình bắt tay (handshake) giữa các bên trong giao tiếp an toàn.

Việc xây dựng các PRF và PRNG mạnh, không thể dự đoán được là nền tảng để đảm bảo tính an toàn cho các hệ thống mật mã hiện đại. Nếu các hàm này bị dự đoán hoặc tái tạo được, toàn bộ hệ thống bảo mật có thể bị phá vỡ. Chính vì thế, các thuật

toán như HMAC (xây dựng PRF từ hàm băm) hay các PRNG dựa trên AES, SHA-2 được sử dụng rộng rãi để đảm bảo tính ngẫu nhiên và an toàn.

CHƯƠNG 2: THIẾT KẾ VÀ CÀI ĐẶT ỨNG DỤNG MINH HỌA

2.1. Yêu cầu hệ thống

Yêu cầu phần cứng:

- Máy chủ: CPU 2 core, RAM 4GB, dung lượng lưu trữ 50GB

- Hệ điều hành: Windows Server/Linux

- Môi trường phát triển: Python 3.x

Yêu cầu phần mềm:

- Cơ sở dữ liệu: Firebase Realtime Database

- Thư viện mã hóa: Bcrypt, PyCryptodome

- Framework: Flask cho backend

- Frontend: HTML5, CSS3, JavaScript với Tailwind CSS và Alpine.js

Yêu cầu bảo mật:

- Mật khẩu được băm bằng thuật toán bcrypt kèm salt

- Chữ ký số sử dụng RSA với độ dài khóa 2048 bit và SHA-256

- Xác thực 2 lớp qua email cho chức năng khôi phục mật khẩu

- Giới hạn kích thước file upload và kiểm tra định dạng

2.2. Mô hình kiến trúc hệ thống

Mô hình 3 lớp:

Presentation Layer (Frontend):

- Giao diện người dùng được xây dựng bằng HTML, CSS và JavaScript

- Sử dụng Tailwind CSS để thiết kế responsive

- Alpine.js cho các component động

- Business Logic Layer (Backend):

- Xử lý bằng Flask (Python)

Các chức năng chính:

- Quản lý phiên đăng nhập

- Xử lý đăng ký/đăng nhập

- Gửi email OTP

- Xử lý chữ ký số

- Quản lý hồ sơ người dùng

Data Access Layer:

- Firebase Realtime Database lưu trữ thông tin người dùng

- Local storage cho file avatar

Luồng dữ liệu chính:

- Người dùng tương tác với giao diện

- Frontend gửi request đến backend qua API

- Backend xử lý và tương tác với Firebase

- Kết quả trả về frontend để hiển thị

2.3. Giao diện người dùng

Trang đăng nhập (login.html):

- Form đăng nhập với username/password

- Link đến trang đăng ký và quên mật khẩu

- Hiển thị lỗi nếu có

- Thiết kế hiện đại với hiệu ứng hover và focus

Trang đăng ký (register.html):

- Form đăng ký với các trường: username, email, password

- Kiểm tra độ mạnh password real-time

- Validate email và password trước khi submit

- Thông báo lỗi chi tiết

Trang quên mật khẩu (forgot_password.html):

- Form nhập email để nhận OTP

- Link quay lại trang đăng nhập

Trang xác minh OTP (verify.html):

- Form nhập mã OTP 6 số

- Bộ đếm ngược thời gian hiệu lực OTP

- Nút gửi lại OTP khi hết hạn

Trang đặt lại mật khẩu (reset_password.html):

- Form nhập mật khẩu mới và xác nhận

- Kiểm tra độ mạnh password real-time

Trang hồ sơ (profile.html):

- Thông tin cá nhân: avatar, họ tên, email

- Form cập nhật thông tin

- Chức năng đổi mật khẩu với xác thực mật khẩu hiện tại

- Upload avatar với kiểm tra kích thước và định dạng

Trang chính (welcome.html):

Giao diện chữ ký số với 3 chức năng chính:

- Tạo khóa (Generate Keys)

- Ký tài liệu (Sign Document)

- Xác thực chữ ký (Verify Signature)

- Hiển thị thông tin người dùng và nút logout

2.4. Mô tả chức năng chính

2.4.1. Đăng ký/ đăng nhập tài khoản người dùng (lưu trữ mật khẩu)

Đăng ký tài khoản:

1. Người dùng nhập thông tin: username, email, password
2. Hệ thống kiểm tra:
    - Email hợp lệ (regex)
    - Password đủ mạnh (8 ký tự, chữ hoa, thường, ký tự đặc biệt)
    - Username và email chưa tồn tại
3. Mật khẩu được băm bằng bcrypt với salt ngẫu nhiên
4. Thông tin lưu vào Firebase:

```
{
  "username": "username",
  "email": "email@example.com",
  "password": "$2a$12$...",  # bcrypt hash
  "first_name": "",
  "last_name": "",
  "gender": "",
  "avatar": "filename.jpg"
}
```

Đăng nhập:

1. Kiểm tra username/password với hash trong database
2. Tạo session nếu hợp lệ
3. Chuyển hướng đến trang chính

Quên mật khẩu:

1. Người dùng nhập email
2. Hệ thống gửi OTP qua email (sử dụng SMTP)
3. Xác minh OTP trước khi cho phép đặt mật khẩu mới

2.4.2. Chữ kí số

Tạo cặp khóa RSA:

1. Sử dụng thư viện JSEncrypt để tạo khóa 2048-bit

2. Hiển thị fingerprint (SHA-256 của public key)
3. Cho phép download private và public key

Ký tài liệu:

1. Người dùng upload tài liệu và private key
2. Tính hash SHA-256 của tài liệu
3. Tạo chữ ký bằng private key
4. Tạo file kết quả dạng JSON chứa:
    - Nội dung gốc
    - Chữ ký
    - Thuật toán
    - Timestamp
5. Cho phép download file đã ký

Xác thực chữ ký:

1. Người dùng upload file đã ký và public key
2. Giải mã chữ ký để lấy hash
3. Tính hash SHA-256 của tài liệu gốc
4. So sánh hai hash và hiển thị kết quả (Valid/Invalid)

CHƯƠNG 3: TỔNG KẾT

3.1. Kết quả đạt được

3.1.1. Về mặt lí thuyết

Nhóm đã hoàn thành báo cáo chuyên sâu về hàm băm (Hash Function), trong đó trình bày rõ ràng và có hệ thống các nội dung sau:

Khái niệm và vai trò của hàm băm trong lĩnh vực an toàn thông tin.

Đặc điểm kỹ thuật và các tính chất quan trọng của một hàm băm tốt như: tính một chiều, tính chống va chạm, và tính xác định.

Phân tích một số thuật toán băm phổ biến như: MD5, SHA-1, SHA-256, và so sánh ưu – nhược điểm giữa các thuật toán này.

Ứng dụng thực tế của hàm băm trong các lĩnh vực như lưu trữ mật khẩu, kiểm tra toàn vẹn dữ liệu, chữ ký số, và blockchain.

Đưa ra các rủi ro, hạn chế và các biện pháp khắc phục trong việc sử dụng hàm băm, đặc biệt là trong bối cảnh tấn công hiện đại như collision attack và preimage attack.

Báo cáo lý thuyết được trình bày khoa học, có minh họa, ví dụ thực tế, kèm theo tài liệu tham khảo đáng tin cậy và đầy đủ.

3.1.2. Về mặt ứng dụng

Nhóm đã phát triển một ứng dụng web minh họa việc sử dụng hàm băm trong chức năng ký số, nhằm giúp người dùng hiểu rõ hơn cơ chế hoạt động và tính ứng dụng của công nghệ này trong bảo mật thông tin.

Giao diện web đơn giản, trực quan, cho phép người dùng:

Đăng nhập, đăng kí lưu mật khẩu.

Tải lên tệp văn bản.

Sinh giá trị băm từ nội dung tệp.

Mô phỏng quá trình ký số và xác minh chữ ký.

Hiển thị kết quả một cách rõ ràng và dễ hiểu.

Một số nội dung triển khai cụ thể:

Ngôn ngữ/libraries sử dụng: Python/Flask (backend), HTML/CSS/JavaScript (frontend), thư viện hashlib để xử lý hàm băm.

Tính năng mô phỏng xác thực chữ ký giúp người dùng hiểu rõ hơn mối liên hệ giữa dữ liệu gốc, giá trị băm và khóa bí mật.

3.1.3. Hạn chế

Mặc dù sản phẩm đạt được các chức năng cơ bản như yêu cầu ban đầu, tuy nhiên vẫn còn một số điểm hạn chế nhất định:

Hệ thống chưa hỗ trợ ký số cho các định dạng tài liệu phổ biến như PDF, DOCX, mà hiện tại mới chỉ xử lý được các tệp văn bản thuần túy (.txt). Điều này làm giảm tính thực tiễn của sản phẩm trong môi trường làm việc chuyên nghiệp, nơi các định dạng tài liệu phức tạp thường được sử dụng.

Giao diện người dùng còn đơn giản, chưa tối ưu về mặt trải nghiệm người dùng (UX/UI).

Chưa tích hợp chức năng quản lý khóa công khai/bí mật một cách đầy đủ như trong các hệ thống thực tế.

3.2. Định hướng phát triển trong tương lai

3.2.1. Kế hoạch ngắn hạn

Trong giai đoạn tiếp theo, nhóm sẽ tập trung vào việc hoàn thiện và mở rộng các tính năng cốt lõi của ứng dụng hiện tại. Cụ thể:

Mở rộng khả năng xử lý định dạng tài liệu:

Triển khai hỗ trợ ký số cho các loại tệp phổ biến như PDF và DOCX, nhằm tăng tính ứng dụng thực tiễn của hệ thống trong môi trường học thuật và doanh nghiệp. Việc này sẽ yêu cầu tích hợp thêm các thư viện xử lý tài liệu như PyMuPDF, python-docx, và xây dựng giao diện phù hợp để hiển thị kết quả.

Tối ưu hiệu năng xử lý hàm băm:

Cải thiện tốc độ xử lý thuật toán SHA-256 đối với các tệp có dung lượng lớn, thông qua kỹ thuật như đọc dữ liệu theo từng khối (chunked reading), sử dụng thư viện tối ưu hơn, hoặc tận dụng đa luồng nếu cần thiết. Điều này nhằm đảm bảo ứng dụng hoạt động mượt mà với cả tài liệu có dung lượng cao.

3.2.2. Kế hoạch dài hạn

Về dài hạn, nhóm định hướng phát triển ứng dụng theo hướng nâng cao bảo mật và mở rộng tính năng, bao gồm:

Tích hợp công nghệ Blockchain:

Áp dụng công nghệ blockchain để lưu trữ các giá trị băm của tài liệu sau khi ký số, từ đó tăng tính minh bạch, bất biến và khả năng truy vết của hệ thống. Điều này đặc biệt hữu ích trong các ứng dụng liên quan đến quản lý văn bản, hợp đồng điện tử, hoặc các hệ thống cần chứng thực nguồn gốc dữ liệu.

Phát triển hệ thống quản lý khóa:

Xây dựng mô-đun quản lý khóa công khai/bí mật cho người dùng, đảm bảo an toàn trong quá trình ký và xác minh chữ ký, đồng thời hướng tới hỗ trợ giao tiếp qua chuẩn PKI (Public Key Infrastructure).

Triển khai hệ thống người dùng và phân quyền truy cập:

Cho phép người dùng đăng ký tài khoản, đăng nhập, và theo dõi lịch sử ký số của riêng mình. Tính năng này sẽ hướng đến xây dựng một hệ sinh thái bảo mật văn bản hoàn chỉnh.

TÀI LIỆU THAM KHẢO

[1]. BrainKart. (n.d.). Hash Functions. Retrieved April 3, 2025,

from https://www.brainkart.com/article/Hash-Functions_8353/

[2]. TutorialsPoint. (n.d.). Cryptography - Hash functions. Retrieved April 3, 2025, from https://www.tutorialspoint.com/cryptography/cryptography_hash_functions.htm

[3]. Wikipedia. (n.d.). Digital signature. Wikipedia. Retrieved April 9, 2025, from https://en.wikipedia.org/wiki/Digital_signature

[4]. GeeksforGeeks. (2021, June 21). RSA and digital signatures. GeeksforGeeks. Retrieved April 9, 2025, from

https://www.geeksforgeeks.org/rsa-and-digital-signatures/

[5]. Nakov, D. (n.d.). RSA signatures. CryptoBook. Retrieved April 10, 2025, from https://cryptobook.nakov.com/digital-signatures/rsa-signatures

[6]. Crypto StackExchange. (2013, March 22). Why hash before signing? Crypto StackExchange. Retrieved April 1, 2025, from

https://crypto.stackexchange.com/questions/12768/why-hash-the-message-before-signing-it-with-rsa

[7]. Microsoft Learn. (n.d.). Cryptographic signatures. Microsoft. Retrieved April 19, 2025, from

https://learn.microsoft.com/en-us/dotnet/standard/security/cryptographic-signatures